Aurora: Neuro-Symbolic AI Driven Advising Agent

Lorena Amanda Quincoso Lugones
lorena.a.quincoso@gmail.com
Florida International University
Miami, FL, United States

Christopher Kverne
ckver001@fiu.edu
Florida International University
Miami, FL, United States

Nityam Sharadkumar Bhimani
nick.bhimani@outlook.com
Northeastern University
Boston, MA, United States

Ana Carolina Oliveira
anamalveira098@gmail.com
Florida International University
Miami, FL, United States

Agoritsa Polyzou
apolyzou@fiu.edu
Florida International University
Miami, FL, United States

Christine Lisetti
lisetti@cs.fiu.edu
Florida International University
Miami, FL, United States

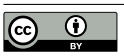
Janki Bhimani
jbhimani@fiu.edu
Florida International University
Miami, FL, United States

Abstract

Academic advising in higher education is under severe strain, with advisor-to-student ratios commonly exceeding 300:1. These structural bottlenecks limit timely access to guidance, increase the risk of delayed graduation, and contribute to inequities in student support. We introduce *Aurora*, a modular neuro-symbolic advising agent that unifies retrieval-augmented generation (RAG), symbolic reasoning, and normalized curricular databases to deliver policy-compliant, verifiable recommendations at scale. Aurora integrates three components: (i) a Boyce-Codd Normal Form (BCNF) catalog schema for consistent program rules, (ii) a Prolog engine for prerequisite and credit enforcement, and (iii) an instruction-tuned large language model for natural-language explanations of its recommendations. To assess performance, we design a structured evaluation suite spanning common and edge-case advising scenarios, including short-term scheduling, long-term roadmapping, skill-aligned pathways, and out-of-scope requests. Across this diverse set, Aurora improves semantic alignment with expert-crafted answers from 0.68 (Raw LLM baseline) to 0.93 (+36%), achieves perfect precision and recall in nearly half of in-scope cases, and consistently produces correct fallbacks for unanswerable prompts. On commodity hardware, Aurora averages 32.99 s per query (vs. 20.13 s for the Raw-LLM baseline), reflecting the runtime overhead of neuro-symbolic grounding. By combining symbolic rigor with neural fluency, Aurora advances a paradigm for accurate, explainable, and scalable AI-driven advising.

CCS Concepts

• **Applied computing** → **Education**; • **Information systems** → *Recommender systems*; • **Computing methodologies** → *Knowledge representation and reasoning*; *Natural language generation.*



This work is licensed under a Creative Commons Attribution 4.0 International License.
SAC '26, Thessaloniki, Greece
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2294-3/2026/03
https://doi.org/10.1145/3748522.3779850

Keywords

Academic advising, Retrieval-Augmented Generation, Neuro-symbolic AI, Large Language Models, Degree Planning

ACM Reference Format:
Lorena Amanda Quincoso Lugones, Christopher Kverne, Nityam Sharadkumar Bhimani, Ana Carolina Oliveira, Agoritsa Polyzou, Christine Lisetti, and Janki Bhimani. 2026. Aurora: Neuro-Symbolic AI Driven Advising Agent. In *The 41st ACM/SIGAPP Symposium on Applied Computing (SAC '26), March 23–27, 2026, Thessaloniki, Greece.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3748522.3779850

1 Introduction

High-quality academic advising plays a vital role in supporting student success, yet it often fails to achieve that goal on many campuses. National surveys [6, 17] reveal that more than 45% of undergraduates cannot meet with their advisor as often as needed, and many report fewer than one interaction per year. In some cases, there are more than 300-600 students per advisor [16], making their workload burdensome, resulting in overscheduled calendars, rushed appointments, and mounting burnout. This advising gap and inadequate guidance result in suboptimal student choices, which, in turn, delay graduation, increase attrition, and undermine trust in institutional support structures.

Research challenge. The core challenge is to design an academic advising system that remains accurate, transparent, and scalable despite the complexity and variability of real-world curricula. However, adaptability to institutional changes is considered a key requirement and critical component of a successful advising system [3]. The key research question, therefore, is how to integrate data-driven language understanding with verifiable symbolic reasoning to produce recommendations that are both explainable and policy-compliant as curricular data evolve over time.

Our approach. To address this advising gap, we present **Aurora**: a modular, reproducible advising framework that integrates Retrieval-Augmented Generation (RAG) with symbolic reasoning. Aurora is a neuro-symbolic AI system that combines the generative flexibility of Large Language Models (LLMs) with the formal rigor

of structured rule-based inference to automate complex aspects of degree planning, particularly those requiring strict adherence to prerequisite structures and dynamic curricular policies. By handling routine, policy-driven advising tasks at scale, Aurora reduces advisor workload and enables more time for personalized, high-value student interactions.

Recent developments in neuro-symbolic AI for education highlight the growing need for hybrid systems that combine the flexibility of deep learning with the transparency and rule enforcement of symbolic reasoning. Neuro-symbolic architectures have been shown to improve trustworthiness and interpretability in educational support systems by grounding neural predictions in explicit curricular or pedagogical knowledge [9, 20]. Parallel work demonstrates that integrating symbolic structures into neural models can enhance student-strategy prediction and decision reliability in learning environments [19]. Most recently, Hooshyar et al. [10] emphasize the importance of responsible, auditable human–AI collaboration in advising and tutoring, noting that purely neural systems struggle to guarantee policy compliance, transparency, or verifiable reasoning. Aurora builds directly on this trajectory by unifying retrieval-augmented LLMs with symbolic rule enforcement to deliver accurate, interpretable, and verifiable degree-planning support at scale.

We contribute the following:

- **A modular neuro-symbolic architecture** for advising that integrates Boyce–Codd Normal Form (BCNF) normalized data storage, SWI-Prolog [22] rule enforcement, and retrieval-augmented prompting with an instruction-tuned LLM.
- **A reproducible benchmark suite** of twenty advising queries spanning short-term scheduling, long-term degree planning, skill-aligned requests, and out-of-scope prompts, each query evaluated over five runs for robust evaluation.
- **Empirical validation** demonstrates that Aurora improves mean cosine similarity from 0.68 (instruction-tuned LLM baseline without retrieval/symbolic) to 0.93 (+36%), achieves perfect precision and recall in about 50 % of the in-scope queries, and reports the runtime overhead introduced by neuro-symbolic grounding.

This study examines three research questions: **RQ1 (Short-term)**, how accurately Aurora recommends next-term courses; **RQ2 (Long-term)**, how effectively it generates prerequisite-compliant roadmaps that minimize time-to-degree; and **RQ3 (Robustness)**, how reliably it handles diverse queries, including skill-aligned and out-of-scope requests, while sustaining accuracy under diverse runtime conditions.

2 Related Work

(i) *Rule-based and static systems.* Early automated advisors were built as flowchart systems or rule-based web applications that checked prerequisites and recommended next courses from static catalogs [15, 21]. A system demonstrated automated prerequisite checking, but this quickly became outdated when requirements shifted [14]. Related expert systems further show the brittleness of hard-coded rules and workflows [4, 5]. Advising requests have been classified and mapped to predefined workflows, while subsequent work enhanced engagement through dialog flow templates and real-time feedback [12, 18]. **Aurora overcomes this brittleness** by storing the catalog in BCNF with explicit keys and relations so that changes propagate cleanly, and by validating recommendations in Prolog, which retains the transparency of rules without hard-coding them into fragile UI workflows.

(ii) *LLM-only systems.* LLMs used as conversational advisors excel at fluency and personalization [2, 11, 23], but without verified catalog grounding, they often hallucinate courses, violate prerequisites, or exceed credit limits. Abdelhamid et al. report these issues when deploying a GPT-4-based advisor at scale [1]. **Aurora eliminates these failure modes** by feeding only vetted course facts and prerequisite chains to the generator and by requiring an explicit fallback when context is insufficient, which prevents hallucinated courses and enforces catalog compliance end to end.

(iii) *Hybrid retrieval and neuro-symbolic reasoning.* Retrieval augmented generation grounds model outputs in institutional data, reducing hallucinations and stale advice, and joint retriever–generator training further reduces factual errors [8, 13]. Neuro-symbolic methods add rule enforcement and transparency to neural fluency [7]. Hybrid architectures that integrate graph-aware reasoning with deep learning can improve multi-semester course sequencing. Yet existing hybrids often lack modularity, rely on ad hoc encodings, or omit strict enforcement of normalization and credit constraints. **Aurora advances this pattern** with a deliberately modular pipeline where retriever, SQL router, Prolog reasoner, and generator are separate services, with declarative rules for explainable enforcement and a normalized catalog that supports consistent credit and prerequisite checks across updates.

Positioning Aurora. Aurora unifies BCNF-normalized catalog storage, SWI-Prolog [22] reasoning over prerequisites and credit caps, and retrieval-augmented prompting [13] to an instruction-tuned LLM. To our knowledge, no prior advising system jointly achieves strict policy compliance through symbolic reasoning, grounding in normalized, update resilient catalog schemas, and modular retrieval augmented prompting for explainable, scalable degree planning. **Empirically, this design outperforms an instruction-tuned LLM baseline without retrieval or symbolic reasoning** on semantic similarity and course level precision and recall while incurring additional runtime overhead, demonstrating improved policy compliance in real advising scenarios.

A parallel line of research in neuro-symbolic educational systems has demonstrated the value of combining deep learning with structured pedagogical knowledge. Recent work shows that symbolic constraints can improve the interpretability and trustworthiness of neural models in tutoring and assessment environments [9]. Complementary approaches infuse expert educational knowledge directly into neural architectures using attention mechanisms to support personalized learning paths [20]. Other studies integrate symbolic reasoning to enhance student-strategy prediction and adaptive decision-making [19]. Most recently, Hooshyar et al. [10] highlight the need for hybrid human–AI systems that maintain accountability and rule compliance in high-stakes educational contexts. While these systems demonstrate the promise of neuro-symbolic designs, none target the unique combination of catalog-grounded

program validation, prerequisite-graph enforcement, and retrieval-augmented planning required for academic advising. Aurora extends this line of research by operationalizing neuro-symbolic reasoning for degree planning at scale, ensuring that every generated recommendation is simultaneously valid, auditable, and aligned with institutional policies.

3 Aurora's Architecture

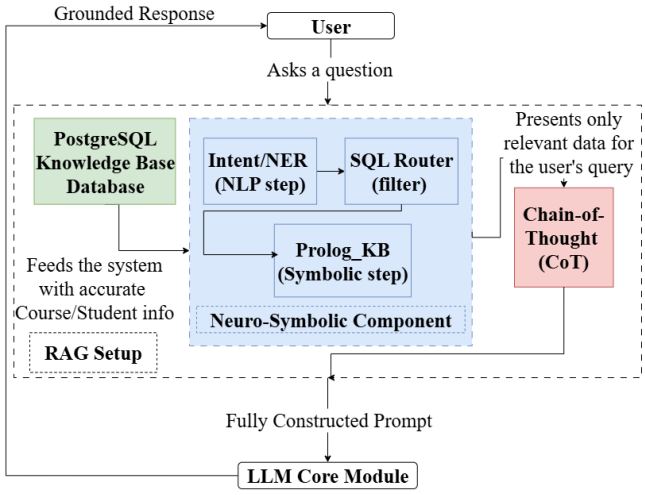


Figure 1: Aurora system overview. Green = knowledge base; blue = neuro-symbolic reasoning; red = Chain-of-Thought prompt segment. Solid arrows denote data flow; dashed line box show control or context boundaries.

Aurora's architecture integrates neural fluency with symbolic rigor to generate degree plans that are linguistically natural and policy-compliant. As shown in Fig. 1, it consists of four key components: a neuro-symbolic reasoning layer, a Chain of Thought (CoT) controller, a relational database interface, and an LLM. Each module handles semantic interpretation, reasoning, validation, and generation, respectively. At the input stage, Aurora uses a dual-stage natural language understanding pipeline: Intent Recognition identifies the advising goal, and Named Entity Recognition (NER) extracts relevant entities like course codes or skills. To improve interpretability and reduce token usage, Aurora employs a structured 5W+1H prompt schema (Who, What, When, Where, Why, How) as a CoT scaffold. The knowledge base is encoded in a relational PostgreSQL schema, ensuring consistency and integrity. For the language layer, Aurora uses **DeepSeek-R1-Distill-Qwen-7B**, chosen for its fluency and lightweight design. Both the database and LLM are modular, allowing for easy substitution. The system workflow starts with the user query being processed through the intent and entity recognizers, guiding the SQL router for targeted database lookups. Retrieved data undergoes symbolic reasoning to enforce constraints, and the final output is generated with the LLM, ensuring fluency and policy compliance.

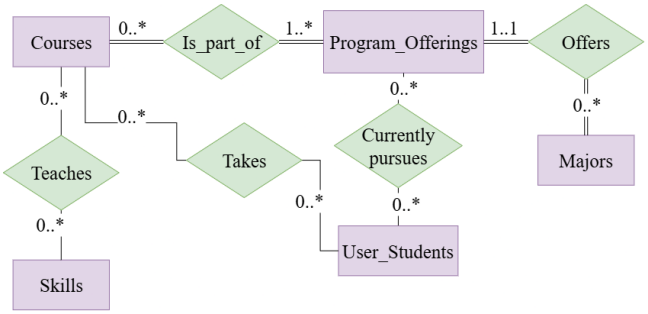


Figure 2: Conceptual ER diagram. Purple = entities; green = relationships. Connectivity encoded in three types of links: 0..* (e.g., a course can cover zero or many skills); 1..* (e.g., a program-offering may include at least one course); 1..1 (e.g., every program-offering is tied to exactly one major).

3.1 PostgreSQL ER Knowledge Base Design

The Conceptual ER Diagram (Fig. 2) forms the structural backbone of Aurora, linking academic and student information to enable personalized, policy-compliant advising. Aurora's advising knowledge base is implemented as a fully normalized entity-relationship (ER) schema in PostgreSQL, connecting students, programs, courses, and skill outcomes. This schema encodes curriculum logic such as prerequisites, degree requirements, and catalog constraints within referentially sound relationships. The design ensures transparency, prevents redundancy, and guarantees consistency even as degree policies evolve. It also directly supports symbolic reasoning tasks like prerequisite traversal and program validation by exposing constraints in a relationally stable format.

PostgreSQL Consistency.
Aurora's catalog back-end was designed directly in Boyce–Codd Normal Form (BCNF). We verified normalization and referential integrity through manual inspection of PostgreSQL's internal catalog tables (`pg_class`, `pg_constraint`, and `pg_attribute`). In every case, functional dependencies hold strictly on declared keys for example:

- **Courses:** `course_id` → credits, department, level, instructor, etc.
- **Program_Course:** (`program_id`, `course_id`) → is_core, recommended_year.
- **User_Program:** (`user_id`, `program_id`) → status, start_date.

No non-key attribute was found to depend on anything other than the designated key, confirming BCNF compliance throughout the schema. This normalization eliminates insertion and update anomalies, ensures deterministic joins, and provides a stable foundation for retrieval and reasoning operations used in the advising workflow.

3.2 Neuro-Symbolic Backend

The design of Aurora's **neuro-symbolic backend** balances flexibility and rigor. Systems that rely *solely on LLMs* are prone to violating prerequisites or credit caps, while *symbolic-only engines* guarantee correctness but lack flexibility when handling the ambiguous

phrasing of real student queries. Aurora reconciles these extremes through a layered division of labor: SQL provides breadth by filtering the curriculum to a compact candidate set, Prolog supplies rigor by validating every academic constraint, and the LLM contributes fluency by phrasing compliant plans in natural language.

3.2.1 Intent, Filtering, and Symbolic Validation. Aurora interprets each query using lightweight intent detection and named-entity recognition to extract program identifiers, course codes, and skill targets. This structured representation guides the SQL router, which deterministically filters the catalog by removing completed courses, selecting eligible options, and applying skill, credit, and term constraints. The resulting candidate set is then passed to the Prolog engine, which enforces all hard academic rules: prerequisites, corequisites, credit caps, and term availability. Only courses satisfying every constraint are retained. This layered design limits the LLM to a small, policy-compliant set of options, ensuring correctness while preserving flexibility in natural-language interpretation.

3.3 Chain-of-Thought Controller and Prompting

Aurora's Chain-of-Thought (CoT) controller converts the verified output of the symbolic backend into a compact prompt for the LLM. Instead of free-form reasoning traces, Aurora uses a structured 5W+1H template that summarizes the advising scenario through the student profile (Who), goal (What), term (When/Where), rationale (Why), and recommended actions (How). Because each field is populated directly from verified course facts and prerequisite chains, the prompt remains concise, interpretable, and grounded in catalog-consistent logic. This structure constrains the LLM to articulate verified recommendations rather than explore unconstrained reasoning paths, reducing hallucinations while preserving natural-language fluency.

Traditional chain-of-thought prompting often yields verbose or inconsistent reasoning traces that are difficult to control in educational domains. The 5W+1H structure offers a natural decomposition of advising intent: *Who* represents the student profile, *What* the target goal or requirement, *When* and *Where* the academic term and program context, *Why* the curricular rationale (e.g., prerequisites or degree milestones), and *How* the recommended action plan. Because these fields map directly to validated entities from the knowledge base, the prompt remains both interpretable and verifiable; each generated statement can be traced back to a specific symbolic fact.

From RAG to structured prompting. The retrieval-augmented context returned by the backend (`COURSE_FACT` and `PREREQ_CHAIN` blocks) populates the relevant 5W+1H fields automatically. This transformation ensures that the model receives all necessary grounding information without redundant examples or long demonstrations. By constraining the reasoning space in this way, Aurora reduces hallucination risk while preserving generative fluency: the LLM can focus on articulation rather than factual recall. The resulting prompt thus functions as a *symbolically informed reasoning frame*, aligning the model's narrative output with verified academic logic.

3.4 LLM Core Module

The final output is generated by the LLM Core Module, which uses the open, instruction-tuned `DeepSeek-R1-Distill-Qwen-7B` model. Quantized for real-time inference on consumer GPUs, the model operates under a strict system directive: respond in two sections (`<think>`, `<response>`), cite only provided evidence blocks, and emit `INSUFFICIENT_CONTEXT` if required data is missing. Controlled decoding (beam search, low temperature) ensures factual consistency and reproducibility.

In summary, together, these modules form a robust, interpretable pipeline: the **Intent & Named Entity Recognition (NER) service** performs precise NLP extraction; the **SQL Router** trims the search space; the **Prolog Reasoner** guarantees correctness; the **CoT Controller** scaffolds generation; and the **LLM Core** delivers natural explanations. By cleanly separating flexible interpretation from symbolic enforcement, Aurora achieves policy-compliant, student-friendly advising grounded in logic, delivered in language.

3.5 Neuro-Symbolic Paradigm Classification

Following the taxonomy proposed by Hooshyar et al. [10], neuro-symbolic AI systems in education can be instantiated through several integration patterns between neural and symbolic components (e.g., Symbolic→Neuro, Symbolic[Neuro], Neuro→Symbolic, Neuro[Symbolic]). These paradigms differ in how knowledge is represented, where learning occurs, and whether symbolic rules are embedded inside neural networks or kept as external reasoning engines. Hooshyar et al. argue that modular designs, in which neural and symbolic subsystems remain separate but exchange intermediate representations, are especially promising for responsible, auditable educational AI [9, 10, 19, 20].

Aurora follows this *hybrid modular neuro-symbolic pipeline* pattern. The neural layer provides semantic interpretation and natural-language generation (NER, embeddings, LLM reasoning), while the symbolic layer enforces institutional rules through SQL filtering and Prolog-based prerequisite and credit-cap validation. The symbolic subsystem constrains the search space *before* neural generation, guaranteeing policy compliance without modifying the underlying model weights. Unlike symbolic-to-neural or neural-to-symbolic integration, Aurora does not embed rules inside the LLM or attempt to learn catalog logic from data; instead, it composes verified symbolic outputs with neural fluency to achieve reliable and interpretable degree-planning recommendations. This places Aurora squarely within the modular neuro-symbolic paradigm advocated for trustworthy educational AI systems [10].

3.6 Limitations of LLMs and Aurora's Mitigations

Large language models exhibit well-known limitations, including hallucination, bias, and opaque internal reasoning. Left unconstrained, they may invent courses, misinterpret catalog policies, or over-generalize from unrelated training data. Aurora mitigates these risks through a strict neuro-symbolic architecture in which the LLM never operates on unverified information: all course facts, prerequisite chains, and credit constraints are produced by SQL and Prolog before generation. The system directive prohibits the model from using external knowledge and requires an explicit

INSUFFICIENT_CONTEXT token when validated evidence is missing, preventing unsupported recommendations. Structured 5W+1H prompting further limits the LLM to articulating verified reasoning rather than generating free-form chains-of-thought. Although Aurora cannot eliminate the intrinsic biases of the underlying model, it confines generation to a symbolically validated search space, reducing opportunities for biased or hallucinated outputs. These safeguards convert the LLM from a black-box decision-maker into a natural-language interface layered on top of transparent, auditable symbolic reasoning.

4 Experimental Setup

4.1 Design Rationale

Evaluating an advising system requires ensuring both rule-compliance and practical usefulness. Because real advising logs require IRB approval and introduce uncontrolled behavioral factors, this study uses a controlled simulation to isolate internal reasoning accuracy under realistic curricular constraints.

4.2 Data and Simulation Environment

We built a reproducible synthetic advising environment covering four representative programs (CS, Data Science, IT, Product Management). Each simulated profile included academic history, degree requirements, and credit limits. Program data were sourced from the institutional catalog, and course–skill mappings followed common taxonomies. While synthetic data lack behavioral realism, they support deterministic benchmarking and isolate reasoning accuracy from human variability.

4.3 Benchmark and Ground Truth

To probe Aurora's reasoning breadth, we designed a 20-query benchmark spanning four advising contexts:

- Short-term scheduling: Selecting next-semester courses given completed history and credit caps.
- Long-term roadmapping: Constructing prerequisite-compliant degree plans.
- Skill-aligned planning: Generating domain-specific tracks (e.g., AI-oriented schedules).
- Out-of-scope recognition: Detecting and declining queries beyond academic policy.

Table 1 shows samples of questions from each category of questions. Each query was paired with an expert reference answer produced independently by two senior students serving in advisor roles, reconciled through consensus. This provided a consistent ground truth for evaluating both factual correctness and explanation quality.

4.4 Metrics and Experimental Setup

Performance was assessed along three complementary dimensions. *Semantic alignment:* cosine similarity between sentence-embedding vectors of Aurora's output and expert responses, serving as a proxy for conceptual coherence and ordering of recommendations. *Course-level accuracy:* precision, recall, and F_1 scores for recommended courses relative to ground truth. *Operational efficiency:* mean runtime per query. These metrics jointly capture whether Aurora (i) reasons consistently with expert logic, (ii) adheres to catalog constraints, and (iii) remains computationally practical for deployment. Experiments were run on a commodity workstation (Intel i9-14900KF CPU, RTX 4070 Ti GPU, 32 GB RAM) to mirror small university-lab conditions. The baseline condition used the same instruction-tuned LLM (DeepSeek-R1-Distill-Qwen-7B) without retrieval or symbolic reasoning, referred to as **Raw-LLM** henceforth. All model parameters were held constant. This isolates the contribution of Aurora's neuro-symbolic pipeline from model or hardware variation.

4.5 LLM Baseline Protocol (Runs & Scoring)

For comparability, the Raw-LLM baseline used the same instruction-tuned model and decoding settings as Aurora's generator, but without retrieval or symbolic reasoning. Every benchmark query was executed across multiple runs on the same hardware, and performance was computed from the extracted course IDs in the model's response (course-level precision, recall, F1), plus cosine similarity of the narrative against the expert rationale, and mean response time per query. This isolates the effect of *neuro-symbolic grounding* (retrieval + SQL + Prolog) from model or hardware differences.

4.6 Implementation details and traceability

4.6.1 SQL filtering. The SQL router trims candidate lists prior to symbolic validation. The exact filter used during experiments is shown in Listing 1.

Listing 1: Filter by program, skill, credit, and taken list

```
SELECT DISTINCT pc.course_id
FROM   program_course pc
LEFT   JOIN course_skill cs USING(course_id)
LEFT   JOIN courses      c  USING(course_id)
WHERE  pc.program_id = %(program)s
  AND (%(has_skills)s AND cs.skill_id = ANY(%(skills)s)
       OR NOT %(has_skills)s)
  AND (%(has_cap)s    AND c.credits <= %(cap)s
       OR NOT %(has_cap)s)
  AND pc.course_id <> ALL(%(taken)s);
```

4.6.2 Prolog knowledge base and greedy planner. Aurora's hard-constraint checker runs in SWI-Prolog. For roadmaps Aurora uses a greedy planner (see Algorithm 1) which packs eligible courses by an "unlock weight" heuristic across semesters; the planner avoids cycles because curricula prerequisites form a DAG.

4.6.3 Prompt assembly and 5W1H framing. After Prolog certifies a candidate set, the router serializes evidence (STUDENT_HISTORY, COURSE_FACT blocks, optional PREREQ_CHAIN), inserts a compact 5W+1H header, and hands the resulting prompt to the generator. An anonymized example prompt body used in evaluation is shown in Listing 2.

4.6.4 Model configuration and system message. Aurora's narrative layer used `DeepSeek-R1-Distill-Qwen-7B` quantized with 4-bit NF4; generation employed a conservative beam-decoding profile. The fixed system directive enforces the two-block output contract and mandates `INSUFFICIENT_CONTEXT` when necessary. An excerpt from the system message is shown in Listing 3.

Listing 3: Excerpt from the fixed system message.

Algorithm 1 Greedy roadmap planner (high level)

Require: *Program*, *Taken*, *CreditCap*, *Start*
Need ← all remaining program courses
Semester ← *Start*; *Plan* ← [ ]
while *Need* ≠ ∅ **do**
  Cap ← credit limit for *Semester*
  Elig ← $c \in Need \mid prereqs_met(c, Taken)$
  if *Elig* = ∅ **then** ▷ nothing unlocked
    Seed ← first course in *Need*
    Pick ← *Seed* plus just enough of its prerequisite *unlockers* to fit *Cap*
  else ▷ some courses unlocked
    rank *Elig* by *unlock weight* = # of future courses each unlocks
    greedily pack highest-weight courses up to *Cap* into *Pick*
    if *Pick* = ∅ **then**
      Pick ← {FIRST(*Elig*)}
    end if
  end if
  add co-requisites to *Pick*
  add any missing prerequisites or allowed alternatives
  trim or pad *Pick* to respect *Cap* and the three-course minimum
  append block(*Semester*, *Pick*) to *Plan*
  Taken ← *Taken* ∪ *Pick*; *Need* ← *Need* \ *Pick*
  Semester ← next academic term
end while
return *Plan*

Listing 2: Illustrative prompt body (anonymized).

```
### STUDENT_QUERY 'I would like a machine-learning schedule next
     spring, max 12 credits.'
### STUDENT_HISTORY ABC1010 DEF2020 GHI3030
### COURSE_FACT id = MLA4100 name = Intro to Machine Learning
     credits = 3 description = 'Supervised and unsupervised basics
     ' id = DST3300 name = Data-Science Tools credits = 3
     description = 'Python, arrays, data frames.' . . . (
     additional vetted courses in the same format)
### PREREQ_CHAIN MLA4100 <- GHI3030, DEF2020 DST <- ABC1010
### 5W1H FRAME Who: B.S. Computer Science What: machine-learning
     schedule next spring, max 12 credits When: Spring 2026 Where:
      n/a Why: machine learning, data science How: using the
     vetted courses above
```

```
### System
You are *Aurora*, an academic-advising assistant [. . .]
Always output exactly two sections:
  1) <think>...</think>
  2) <response>...</response> beginning with
     'As your academic advisor, I recommend...'
Use only the supplied context [. . .]
If you need data you don't see, reply exactly:
INSUFFICIENT_CONTEXT
```

4.6.5 Interpretability and provenance. Every generated recommendation is accompanied by provenance: the SQL used to produce candidates, the Prolog trace that validated the set, and the evidence blocks passed to the LLM. This full-stack trace enables reproducibility, audits, and fine-grained debugging crucial for institutional adoption.

5 Results: Aurora-at-work

Figure 3 shows per-query mean cosine similarity between system outputs and ground-truth answers. Higher values indicate greater semantic alignment with handcrafted expert responses.

Across all in-scope benchmark queries, Aurora achieves a mean cosine similarity of 0.93 compared to 0.68 for the Raw-LLM baseline, indicating substantially closer alignment with expert-crafted advising recommendations. Course-level precision, recall, and F_1 scores average 0.81–0.83, whereas the unconstrained LLM frequently proposes ineligible or nonexistent courses, resulting in near-zero accuracy. Aurora introduces additional end-to-end latency due to SQL pruning and Prolog validation before generation. This ensures that the LLM focuses solely on phrasing verified decisions rather than performing unconstrained reasoning. Out-of-scope prompts reliably trigger the fallback template, demonstrating controlled behavior and policy-compliant robustness.

The planning and generation pipeline averages 32.99 s per query (vs. 20.13 s Raw-LLM) on the same hardware, reflecting the cost of precise symbolic enforcement.

Table 1: Representative evaluation queries.

Category	Persona	Sample Query
Short-term (Next Term)	CS Major	What courses should I take next semester to stay on track for my CS-BS degree?
Long-term (Degree Plan)	CS Major	Plan the rest of my CS-BS degree so I can graduate on time.
Skill-aligned	CS Major	I'd like an AI-oriented schedule for next Fall. What should I take?
Short-term (Next Term)	CS Minor	Please suggest my next-term schedule to finish the CS minor (12-credit cap).
Out-of-scope	CS Major	What can I do if a class is difficult?

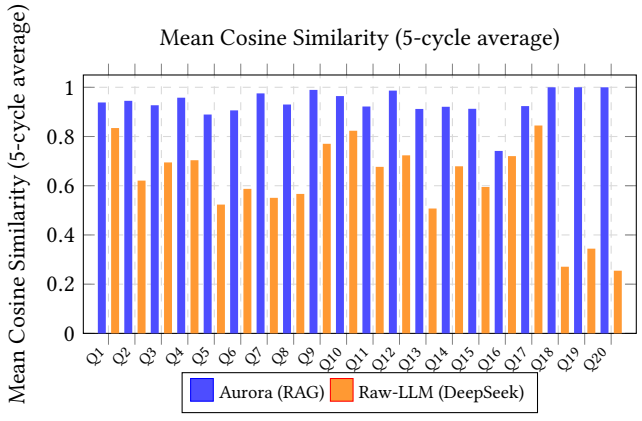


Figure 3: Per-query cosine similarity averaged over five runs.

5.1 Validation of System Consistency and Prompt Efficiency

To confirm Aurora's robustness and architectural soundness, we performed a two-part sensitivity analysis focusing on (i) the relational data layer and (ii) the retrieval–generation (RAG) footprint. Together, these tests verify that the system's foundations are both theoretically consistent and practically efficient under realistic advising workloads.

RAG Footprint Analysis.
Retrieval-augmented generation (RAG) constrains Aurora's prompt size by fetching only the minimal catalog subset relevant to each query. Without retrieval, the system would serialize the entire course catalog into the model prompt approximately $210 \times 60 \approx$ 12,600 tokens. Empirically, across the benchmark suite, Aurora's retriever produces a mean prompt size of 492 ± 444 tokens per query (range: 0–1,346) and retrieves on average 7.8 ± 8.7 courses (range: 0–24).

The observed reduction ratios are:

$$\text{Worst case} \quad \frac{1{,}346}{12{,}600} \approx 0.11\ (11\%)$$

$$\text{Typical} \quad \frac{492}{12{,}600} \approx 0.04\ (4\%)$$

Simple or out-of-scope queries yield zero-length prompts, confirming that the early-exit logic is functioning as intended. By logging $\langle \text{queryID}, N_{\text{retrieved}}, |\text{tokens}_{\text{RAG}}| \rangle$ for every inference call, Aurora maintains full auditability of retrieval behavior and prompt expansion. Overall, this analysis confirms that RAG consistently reduces token utilization by over an order of magnitude while preserving completeness and recall, ensuring scalability and responsiveness in real-world academic advising contexts.

5.2 Takeaways from the Evaluation

Accuracy and Reliability. Aurora improves semantic alignment from 0.68 to 0.93 (+36%) and achieves perfect precision and recall on roughly half of all in-scope queries, demonstrating that symbolic grounding yields verifiably correct degree plans.

Robustness and Policy Compliance. Out-of-scope prompts consistently trigger explicit fallbacks, ensuring that uncertainty is surfaced rather than glossed over, which is critical for institutional deployment.

Efficiency and Scalability. SQL filtering and Prolog validation are lightweight individually, but the full pipeline averages 32.99 s per query. Because the LLM receives only compact, validated context, generation (rather than reasoning) dominates runtime.

Traceability and Explainability. Every recommendation is fully auditable from SQL retrieval to Prolog validation and the final prompt, supporting reproducibility and future advisor oversight.

Broader Significance. Taken together, these results position Aurora as a concrete demonstration that neuro-symbolic architectures can make large language models *operationally trustworthy* in educational decision-making. By achieving near-expert accuracy, safe fallbacks, and transparent reasoning on commodity hardware, Aurora transforms LLM-based advising from a proof-of-concept into a viable institutional tool. More broadly, the system illustrates how symbolic grounding can turn generative models into dependable collaborators, not replacements, for human advisors, enabling scalable, explainable support for students while preserving academic oversight.

6 Discussion

6.1 Interpretation of Findings

Aurora's evaluation demonstrates that combining symbolic reasoning with neural generation can substantially improve internal reasoning fidelity in automated academic advising. The observed gains in semantic alignment (from 0.68 to 0.93 cosine similarity) and balanced precision–recall confirm that grounding an LLM in structured curricular logic reduces factual errors and enforces prerequisite compliance. These results highlight an important methodological insight: large language models can reason more reliably when constrained by explicit, verifiable rules rather than free-text prompts alone. At the same time, these findings should be interpreted within the limits of a controlled simulation. The experiment validates the architectural integrity of Aurora: its ability to produce consistent, policy-compliant outputs, not its educational impact. Because the test data were synthetically generated and devoid of human variability, these results speak to internal validity rather than to student learning outcomes or advising satisfaction. In this sense, the work provides evidence of technical reliability, not proof of pedagogical effectiveness. The improvements over an instruction-tuned LLM baseline (Raw-LLM) also underscore a broader research point: retrieval and symbolic verification do not merely add interpretability but actively shape reasoning behavior. Aurora's architecture thus serves as a replicable framework for other educational AI systems where correctness, auditability, and transparency are critical.

6.2 Educational and Practical Implications

Aurora demonstrates how neuro-symbolic methods can improve transparency and accountability in advising systems. The provenance trail (from SQL filtering to Prolog validation) supports auditability and provides advisors with clear insight into how recommendations were produced. Current latency reflects the overhead of modular neuro-symbolic orchestration. Aurora is therefore best understood as a co-advising assistant: it automates rule enforcement and data retrieval, while human advisors supply contextual judgment and mentoring. Integrating Aurora into existing workflows will require interface design that surfaces explanations effectively and training that helps advisors interpret AI-supported guidance.

6.3 Limitations and Future Directions

The present study's primary limitation lies in its reliance on simulated student data. While this approach enabled reproducible testing of reasoning accuracy, it cannot capture the behavioral, motivational, or affective dynamics of real advising conversations. Consequently, Aurora's high precision and recall reflect compliance with curricular logic, not demonstrable improvement in student outcomes. A second limitation is interactional realism: the current evaluation omits live user interaction and human–AI collaboration effects. Factors such as trust, interpretability, and cognitive load were not measured. Additionally, the benchmark represents a single institution's catalog and policy framework; broader generalization across diverse universities would require adapting both schema design and rule sets. Future research should therefore pursue (i)

field validation with real advisors and students, assessing not only accuracy but also usability and trust; (ii) user-interface design to enhance explainable feedback and ensure high usability based on end-user feedback, (iii) comparative studies examining how advisors interpret or override Aurora's recommendations; (iv) expanded reasoning coverage, extending beyond prerequisite graphs to include electives, double majors, and institutional exceptions; and (v) fairness and bias analysis, ensuring equitable treatment across academic pathways.

7 Conclusion

This study presented Aurora, a neuro-symbolic advising system that integrates retrieval-augmented generation with rule-based reasoning to deliver verifiable, policy-compliant course recommendations. By grounding large-language-model outputs in normalized curricular data and enforcing prerequisite and credit constraints through symbolic logic, Aurora achieves high semantic accuracy with measurable runtime overhead. Within a controlled, simulated evaluation, Aurora outperformed an instruction-tuned LLM baseline on all major advising tasks, providing credible evidence that LLMs, when equipped with highly specialized and context-constrained reasoning layers, can produce substantially more reliable results than unconstrained generative systems. While further validation with real students and authentic advising data remains necessary, Aurora already demonstrates how verifiable neuro-symbolic architectures can move AI-driven advising from experimental prototypes toward practical, auditable tools for higher education and lay the foundation for future human-in-the-loop deployments that emphasize trust, usability, and educational impact.

Acknowledgments

This work was supported in part by the National Science Foundation under grants CSR-2402328, CAREER-2338457, CSR-2406069, CSR-2323100, and HRD-2225201. We acknowledge Dr. Gregory Murad Reis for his early insights on integrating RAG methods with LLM-driven reasoning, which helped reinforce the viability of our system design approach. The source code, dataset schema, and evaluation suite for Aurora is publicly available at https://github.com/Damrl-lab/Aurora.

References

[1] Sherif Abdelhamid, James Bangura, and Shahryar Shah. 2025. Advisely: AI-Powered Academic Advising Using Large Language Models. In *Proceedings of the 14th Int. Conf. on New Perspectives in Science Education*. Florence, Italy. Demonstrates GPT-4 advising but notes prerequisite violations without rule checks.

[2] Angel Aguila, Nhan Tran Ngoc, Nam Anh Dang Nguyen, Kha-Tu Huynh, An Mai, Tan Duy Le, and Nguyen Tan Viet Tuyen. 2024. Large Language Model in Higher Education: Leveraging Llama2 for Effective Academic Advising. In *2024 International Conference on Advanced Technologies for Communications (ATC)*. 590–595. doi:10.1109/ATC63255.2024.10908321

[3] Samuel Bisaso, William Wasswa, and Gilbert Muhumuza. 2025. Towards the Application of an NLP-driven Conversational Model for Efficient and Affective Career Guidance for Students: A Scoping Review. *European Journal of Applied Science, Engineering and Technology* 3, 3 (2025), 240–253.

[4] Olawande Daramola, Onyeka Emebo, Ibukun Afolabi, and Charles Ayo. 2014. Implementation of an intelligent course advisory expert system. *International Journal of Advanced Research in Artificial Intelligence* 3, 5 (2014), 6–12.

[5] Gökhan Engin, Burak Aksoyer, Melike Avdagic, Damla Bozanlı, Umutcan Hanay, Deniz Maden, and Gurdal Ertek. 2014. Rule-based expert systems for supporting university students. *Procedia Computer Science* 31 (2014), 22–31. doi:10.1016/j.procs.2014.05.230

[6] Colleen Flaherty. [n. d.]. Student survey reveals gaps in core academic advising functions — insidehighered.com. https://www.insidehighered.com/news/student-success/academic-life/2023/02/28/student-survey-reveals-gaps-core-academic-advising. [Accessed 30-09-2025].

[7] Artur d'Avila Garcez and Luis C Lamb. 2023. Neurosymbolic ai: The 3 rd wave. *Artificial Intelligence Review* 56, 11 (2023), 12387–12406.

[8] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. Retrieval augmented language model pre-training. In *International conference on machine learning*. PMLR, 3929–3938.

[9] Danial Hooshyar, Roger Azevedo, and Yeongwook Yang. 2024. Augmenting Deep Neural Networks with Symbolic Educational Knowledge: Towards Trustworthy and Interpretable AI for Education. *Machine Learning and Knowledge Extraction* 6, 1 (2024), 593–618.

[10] Danial Hooshyar, George Šír, Yiming Yang, Eve Kikas, Raija Hämäläinen, Tommi Kärkkäinen, and Roger Azevedo. 2025. Towards responsible AI for education: Hybrid human-AI to confront the Elephant in the room. *arXiv preprint arXiv:2504.16148* (2025).

[11] Md Akib Zabed Khan, Agoritsa Polyzou, and Neila Bennamane. 2022. How Can We Use LLMs for EDM Tasks? The Case of Course Recommendation. In *Joint Proceedings of the Human-Centric eXplainable AI in Education and the Leveraging Large Language Models for Next Generation Educational Technologies Workshops (HEXED-L3MNGET 2024) co-located with EDM 2024*. CEUR-WS.

[12] Mohammad Amin Kuhail, Haseena Al Katheeri, Joao Negreiros, Ahmed Seffah, and Omar Alfandi. 2023. Engaging students with a chatbot-based academic advising system. *International Journal of Human–Computer Interaction* 39, 10 (2023), 2115–2141.

[13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 9459–9474.

[14] Oge Marques, Xundong Ding, and Sam Hsu. 2001. Design and development of a Web-based academic advising system. In *31st Annual Frontiers in Education Conference. Impact on Engineering and Science Education. Conference Proceedings (Cat. No. 01CH37193)*, Vol. 3. IEEE, S3C–6. doi:10.1109/FIE.2001.964071

[15] W Scott Murray and Louis A Le Blanc. 1995. A decision support system for academic advising. In *Proceedings of the 1995 ACM symposium on Applied computing*. 22–26.

[16] NACADA Journal. 2025. Defining and Institutionalizing Proactive Advising. *NACADA Journal* 44, 2 (2025), 83–98.

[17] National Survey of Student Engagement. 2021. *Academic Advising Topical Module Report*. Technical Report. Indiana University Center for Postsecondary Research. Shows median <2 advising meetings per student per year.

[18] Amin Y Noaman and Fekry Fouad Ahmed. 2015. A new framework for e academic advising. *Procedia Computer Science* 65 (2015), 358–367. doi:10.1016/j.procs.2015.09.124

[19] Anup Shakya, Vasile Rus, and Deepak Venugopal. 2021. Student Strategy Prediction Using a NeuroSymbolic Approach. In *International Educational Data Mining Society*.

[20] Ange Tato and Roger Nkambou. 2022. Infusing Expert Knowledge Into a Deep Neural Network Using Attention Mechanism for Personalized Learning Environments. *Frontiers in Artificial Intelligence* 5 (2022), 921476. doi:10.3389/frai.2022.921476

[21] William E Wehrs. 1992. Using an expert system to support academic advising. *Journal of Research on Computing in Education* 24, 4 (1992), 545–562.

[22] Jan Wielemaker, Tom Schrijvers, Markus Triska, and Torbjörn Lager. 2012. Swi-prolog. *Theory and Practice of Logic Programming* 12, 1-2 (2012), 67–96.

[23] Thorsten Zylowski, N Sautchuk Patricio, Wladimir Hettmann, Katharina Anderer, Matthias Wölfel, and PA Henning. 2024. Evaluating trustworthiness, usability and explainability of an educational pathway recommendation system that uses a large language model. In *EDULEARN24 Proceedings*. IATED, 6461–6470.